

Rekurrent neyron tarmoqlar (RNN)

Tabiiy tilni qayta ishlash — 7-ma'ruza

A.A. Abdulali

11:00–12:20 · mavzu №7

25-iyun 2026

Prezentatsiya rejasi

- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat
- 4 Vaqt bo'ylab yoyilish va BPTT
- 5 Til modellashtirish va matn tasniflash
- 6 Xulosa va 7-amaliyotga ko'prik

Takrorlash: o'tgan darslardan ikki savol

1-savol

5-ma'ruzada bigram faqat 1 oldingi so'zga qaradi. “men kitobni do'stinga _____” — kesim uzoqda. O'zgarmas oyna uzoq bog'liqlikni ko'ra oladimi?

2-savol

6-amaliyotda Word2Vec har so'zga vektor berdi. Lekin gap — so'zlar ketma-ketligi. Word2Vec kabi statik embedding o'zi tartibni va xotirani saqlaydimi?

Takrorlash: o'tgan darslardan ikki savol

1-savol

5-ma'ruzada bigram faqat 1 oldingi so'zga qaradi. “men kitobni do'stinga _____” — kesim uzoqda. O'zgarmas oyna uzoq bog'liqlikni ko'ra oladimi?

2-savol

6-amaliyotda Word2Vec har so'zga vektor berdi. Lekin gap — so'zlar ketma-ketligi. Word2Vec kabi statik embedding o'zi tartibni va xotirani saqlaydimi?

1-javob

Yo'q. O'zgarmas oyna faqat $n-1$ so'zni ko'radi; undan uzoqdagi kesim ko'rinmaydi (Markov cheklovi).

2-javob

Yo'q. Statik embeddinglarda har vektor kontekstdan mustaqil — tartib yo'qoladi. Bugun xotirali ketma-ketlik modeli quramiz: **RNN**.

Bugungi dars maqsadlari

- 1 Nega o'zgarmas oyna (N-gram / oldinga uzatuvchi til modeli) uzoq bog'liqlikni barqaror ushlay olmasligini **tushuntira olasiz**.
- 2 RNN (*Recurrent Neural Network*) bir qadamini — yashirin holatni $h_1 = [0.762, 0.000]$ ga — qo'lda **hisoblay olasiz**.
- 3 Vaqt bo'ylab yoyilish (*unrolling*), og'irliklar ulashilishi va BPTT (*Backpropagation Through Time*) g'oyasini **tushuntira olasiz**.
- 4 RNN ni til modellashtirish va matn tasniflashda (ko'p-dan-bir) **qo'llay olasiz**.

Oldingi bilimlar: 5-ma'ruzadan til modeli va Markov taxmini; 6-ma'ruzadan embedding (kirish vektori), softmax, gradient. Matematikadan: matritsa-vektor ko'paytma, tanh, zanjir qoidasi.

Bo'lim	Mavzu	Natija
1	O'zgarmas oynadan ketma-ket xotiraga	oyna cheklovi tushuniladi
2	RNN konsepsiyasi: yashirin holat	$h_1 = [0.762, 0.000]$
3	Vaqt bo'ylab yoyilish va BPTT	$h_2 = [0.642, 0.762]$
4	Til modeli va matn tasniflash	softmax orqali bashorat
–	Xulosa va 7-amaliyotga ko'prik	amaliyot yo'nalishi

Asosiy yo'nalish: N-gram va statik embeddinglardan rekurrent ketma-ketlik modeliga o'tamiz.

Bugungi muammo: gap uzunligi har xil, kesim uzoqda

Muammo: o'zgarmas kontekst oynasi

Oldinga uzatuvchi til modeli faqat oxirgi k ta so'zni ko'radi.

Masalan:

men kitobni do'stimga berdim

gapida berdim ni bashoratlash uchun uzoqdagi men ham muhim bo'lishi mumkin.

Agar k kichik bo'lsa, uzoq kontekst modelga kirmaydi. Agar k kattalashsa, kirish o'lchami $k \cdot d$ bo'lib, parametrlar soni ham ortadi.

Xulosa: model baribir oldindan tanlangan **chegaralangan oynaga** bog'liq qoladi.

Bugungi yechim: RNN

- 1 So'zlarni **birma-bir** o'qiymiz.
- 2 Har qadamda **yashirin holat** (*hidden state*) yangilanadi.
- 3 Holat oldingi so'zlar haqidagi **vektorli xulosani** saqlaydi.

RNN (*Recurrent Neural Network*) bir xil yangilash amalini ketma-ketlik bo'ylab takrorlaydi va o'zgaruvchan uzunlikdagi gapni o'zgarmas o'lchamli holatda jamlaydi.

Prezentatsiya rejasi

- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat
- 4 Vaqt bo'ylab yoyilish va BPTT
- 5 Til modellashtirish va matn tasniflash
- 6 Xulosa va 7-amaliyotga ko'prik

Intuitsiya: o'zgarmas oyna — tor ko'rish maydoni

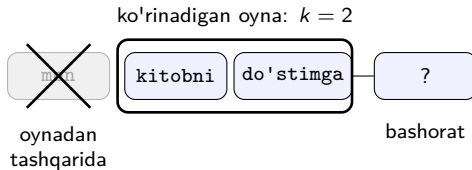
O'zgarmas oyna degani: model har safar faqat oxirgi k ta so'zni ko'radi.

men kitobni do'stinga → ?

Bu yerda $k = 2$: model faqat kitobni va do'stinga ni ko'radi.

Muammo: oynadan tashqarida qolgan so'z modelga ko'rinmaydi. Shuning uchun uzoq bog'liqlik yo'qoladi.

N-gram *sanaydi*, neyron model *o'rganadi*; lekin ikkalasi ham **o'zgarmas kontekst oynasiga** bog'liq.



Intuitsiya: model faqat oyna ichidagi so'zlarni ko'radi. Bizga esa ketma-ketlik bo'ylab yangilanadigan **xotira** kerak.

Oldinga uzatuvchi neyron til modeli: rasmiy ta'rif

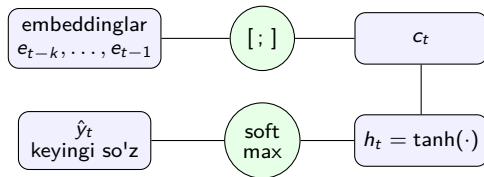
$$c_t = [e_{w_{t-k}}; \dots; e_{w_{t-1}}] \in \mathbb{R}^{kd}$$

$$h_t = \tanh(W_h c_t + b_h), \quad \hat{y}_t = \text{softmax}(W_o h_t + b_o).$$

Belgilashlar

- e_w — w so'zining embeddingi;
- $[;]$ — vektorlarni birlashtirish;
- c_t — kd o'lchamli kirish;
- h_t — yashirin qatlam;
- $\hat{y}_t$ — keyingi so'z ehtimollari.

Cheklov: model kirishi doim kd o'lchamli. Demak, u faqat belgilangan uzunlikdagi kontekstni oladi.



birlashtirish → yashirin qatlam → ehtimol

Intuitsiya: bu model oyna ichidagi embeddinglarni bitta vektorga birlashtirib, keyingi so'zni bashorat qiladi.

Keltirib chiqarish: nega o'zgarmas oyna uzoq bog'liqlikni ushlay olmaydi?

1-qadam. Oldinga uzatuvchi model bashorati faqat oxirgi k ta so'zga bog'liq:

$$c_t = [e_{w_{t-k}}; \dots; e_{w_{t-1}}].$$

Demak, k so'zdan uzoqdagi ma'lumot kirishga tushmaydi.

2-qadam. O'zbekcha SOV(Subject Object Verb) tartibida kesim ko'pincha gap oxirida keladi:

men kitobni do'stinga berdim.

Agar $k = 2$ bo'lsa, berdim ni bashoratlashda model faqat kitobni va do'stinga ni ko'radi; men oynadan tashqarida qoladi.

3-qadam. k ni oshirish ham to'liq yechim emas: kirish o'lchami kd bo'ladi, birinchi vazn matritsasidagi parametrlar soni ham k bilan ortadi. Baribir maksimal kontekst uzunligi oldindan chegaralangan bo'lib qoladi.

Xulosa: kerakli mexanizm — o'zgarmas oyna emas, o'tmishni ixcham holatga jamlab boradigan **xotira**. Bu g'oya RNN da ishlatiladi.

Qo'lda misol: kontekst vektorini birlashtirish

Berilgan embeddinglar:

$$e_{\text{men}} = [0.5, 0.3], \quad e_{\text{kitobni}} = [0.2, 0.6].$$

Bu yerda kontekst oynasi $k = 2$, har bir embedding o'lchami $d = 2$.

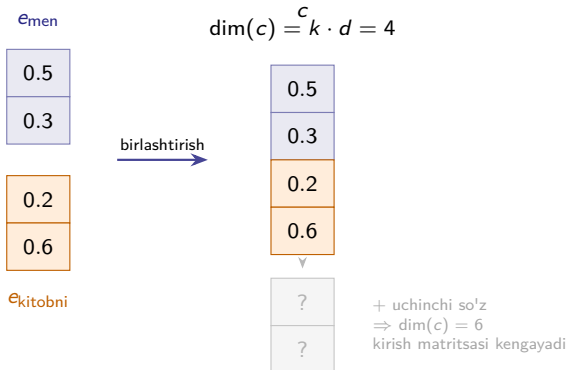
Birlashtirish:

$$c = [e_{\text{men}}; e_{\text{kitobni}}]$$

$$c = [0.5, 0.3, 0.2, 0.6].$$

$$\dim(c) = k \cdot d = 2 \cdot 2 = 4.$$

Nega bu muhim? k oshsa, c uzunlashadi; demak, birinchi vazn matritsasining kirish o'lchami ham oshadi. Shuning uchun oyna oldindan tanlangan va cheklangan bo'lib qoladi.



Intuitsiya: oldinga uzatuvchi model kontekstni bitta uzun vektorga yopishtiradi; RNN esa keyingi slaydlarda bu muammoni holat orqali yumshatadi.

Sizning vazifangiz: yana bir kontekst vektorini birlashtirish

Vazifa

Berilgan embeddinglar:

$$e_u = [0.1, 0.9], \quad e_{keldi} = [0.4, 0.4].$$

Kontekst oynasi $k = 2$ bo'lsa,

$$c = ? \quad \dim(c) = ?$$

Eslatma: $[;]$ belgisi vektorlarni ketma-ket birlashtirishni bildiradi.

Sizning vazifangiz: yana bir kontekst vektorini birlashtirish

Vazifa

$$e_u = [0.1, 0.9], \quad e_{keldi} = [0.4, 0.4], \quad k = 2.$$
$$c = ? \quad \dim(c) = ?$$

Javob

$$c = [e_u; e_{keldi}] = [0.1, 0.9, 0.4, 0.4].$$

$$\dim(c) = k \cdot d = 2 \cdot 2 = 4.$$

Agar oynaga yana bitta so'z kiritish uchun k ni 1 ga oshirsak, c uzunligi yana d ga ortadi. Demak, o'zgarmas oynali oldinga uzatuvchi model uzun kontekstga tabiiy moslasha olmaydi.

Kod va formula: o'zgarmas oynali kirish

Python kodi

```
import numpy as np

e_men = np.array([0.5, 0.3])
e_kitobni = np.array([0.2, 0.6])

# o'zgarmas oyna: embeddinglarni birlashtirish
c = np.concatenate([e_men, e_kitobni])
print(c, c.shape)      # [0.5 0.3 0.2 0.6] (4,)

# m = yashirin qatlam o'lchami
m = 10
W = np.random.randn(m, c.shape[0])

# formula bo'yicha chiziqli qadam
z = W @ c
print(W.shape, z.shape) # (10, 4) (10,)
```

Kod va formula mosligi

Kod	Formula
concatenate	$[e_1; e_2]$
c.shape[0]	$k \cdot d$
W.shape	$m \times (kd)$
W @ c	Wc

Asosiy nuqta: o'zgarmas oynali modelda k oldindan tanlanadi. Shuning uchun c uzunligi $k \cdot d$ ga, W ning ustunlar soni esa aynan shu qiymatga bog'lanadi.

Agar k o'zgarsa, c ham, W shakli ham o'zgaradi. Keyingi g'oya: RNN kontekstni uzun vektorga yopishtirmay, holatni bosqichma-bosqich yangilaydi.

O'zgarmas oyna tuzog'i: ikki yomon tanlov

Murosa

- **Kichik** k : uzoqdagi so'zlar oynadan tashqarida qoladi.
- **Katta** k : kirish vektori va parametrlar soni ortadi.
- Har ikkala holatda ham maksimal kontekst uzunligi **oldindan chegaralangan**.

$$c \in \mathbb{R}^{kd}, \quad W \in \mathbb{R}^{m \times kd}$$

k oshsa, c uzunlashadi va W ning ustunlar soni ham ortadi.

Yechim g'oyasi: o'tmishni uzun vektorga yopishtirish emas, balki uni o'zgarmas o'lchamli **holat**ga jamlab borish kerak.

Keyingi bo'lim: RNN

RNN har qadamda eski holat va joriy so'zdan yangi holat hosil qiladi:

$$h_t = f(h_{t-1}, x_t).$$

Og'irliklar barcha qadamlarda **ulashilgan** (*shared weights*) bo'ladi.

Prezentatsiya rejasi

- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat**
- 4 Vaqt bo'ylab yoyilish va BPTT
- 5 Til modellashtirish va matn tasniflash
- 6 Xulosa va 7-amaliyotga ko'prik

Intuitsiya: o'qiyotganda “xayolda” xulosa saqlaymiz

Oddiy intuitsiya. Gapni o'qiyotganda har so'zdan keyin tushunchamizni **yangilaymiz**; boshidan qayta o'qimaymiz.

RNN ham shunday ishlaydi:

h_t — shu paytgacha o'qilganlarning ixcham xulosasi.

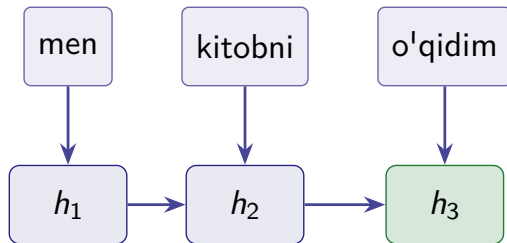
Bu yerda h_t **yashirin holat** (*hidden state*) deyiladi.

Eslatma: “xotira” deganda inson xotirasi emas, oldingi qadamlar haqidagi *vektorli xulosa* nazarda tutiladi.

Har qadamda yangi holat oldingi holat va joriy so'zdan hosil bo'ladi:

$$h_t = f(h_{t-1}, x_t).$$

birma-bir o'qiymiz



butun gap
xulosasi

Nega “rekurrent”? Bir xil yangilash amali va bir xil og'irliklar (W_h , W_x) har qadamda takrorlanadi. Shu sababli model turli uzunlikdagi ketma-ketliklarni qayta ishlay oladi.

RNN rekurrensiyasi: rasmiy ta'rif

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

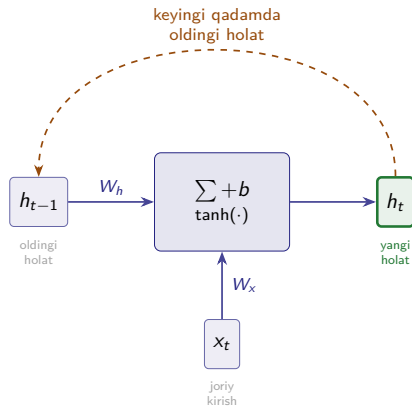
$$x_t \in \mathbb{R}^d, \quad h_t \in \mathbb{R}^m, \quad W_x \in \mathbb{R}^{m \times d}, \quad W_h \in \mathbb{R}^{m \times m}, \quad b \in \mathbb{R}^m.$$

Belgilashlar

- x_t — t -qadamdagi kirish, ya'ni so'z embeddingi;
- h_t — yashirin holat (*hidden state*): oldingi qadamlar haqidagi vektorli xulosa;
- W_h — holat→holat, W_x — kirish→holat og'irliklari;
- b — siljitish hadi (*bias*), $\tanh$ — nochiziqli funksiya;
- odatda $h_0 = \mathbf{0}$ dan boshlanadi.

Asosiy nuqta: W_h va W_x barcha vaqt qadamlarda bir xil (*shared weights*) ishlatiladi.

Har bir arrow formuladagi bitta hadning yo'lini ko'rsatadi.



Holat vektorining o'lchami m gap uzunligidan mustaqil. Shu sababli RNN turli uzunlikdagi ketma-ketliklarni bir xil qoida bilan qayta ishlaydi.

Keltirib chiqarish: rekurrensiya qayerdan keladi?

1-qadam: maqsad

Butun o'tmishni

$$x_1, x_2, \dots, x_t$$

o'zgarmas o'lchamli holatga jamlash kerak:

$$h_t \in \mathbb{R}^m.$$

2-qadam: rekursiv g'oya

Yangi holat eski holat va joriy kirishdan hosil bo'ladi:

$$h_t = f(h_{t-1}, x_t).$$

Bu yerda h_{t-1} — oldingi so'zlargacha bo'lgan xulosa.

3-qadam: o'rganiladigan funksiya

f ni sodda neyron ko'rinishda olamiz:

chiziqli birikma + nochiziqlilik.

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b).$$

Xulosa: h_t rekursiya orqali $x_1, \dots, x_t$ dan ta'sir olgan **ixcham vektorli xulosadir**. Bu yondashuvda oldindan belgilangan o'zgarmas kontekst oynasi yo'lq.

$\tanh$ har bir koordinatani $[-1, 1]$ oralig'ida ushlaydi.

Qo'lda hisob: RNN bir qadami — $h_1 = [0.762, 0.000]$

Berilganlar:

$$W_h = W_x = I_2 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad b = \mathbf{0}.$$

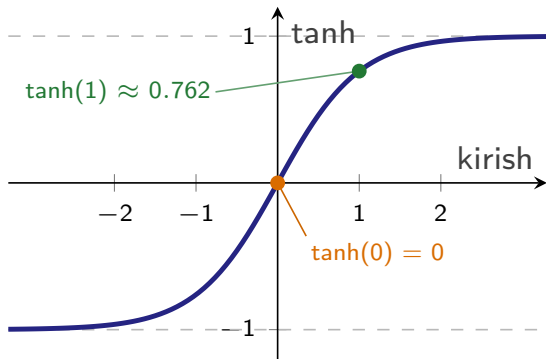
$$h_0 = [0, 0], \quad x_1 = [1, 0].$$

Bu soddalashtirilgan o'quv misoli. Amalda W_h va W_x o'qitish jarayonida o'rganiladi.

Hisoblaymiz:

$$\begin{aligned} h_1 &= \tanh(W_h h_0 + W_x x_1 + b) \\ &= \tanh([0, 0] + [1, 0] + [0, 0]) \\ &= [\tanh(1), \tanh(0)] \\ &= [\mathbf{0.762}, \mathbf{0.000}]. \end{aligned}$$

$\tanh(1) \approx 0.7616$, $\tanh(0) = 0$. Demak, birinchi kirish holatda **vektorli iz** qoldirdi.



Amaliyotga ko'prik:

```
np.allclose(h1, [0.762, 0.0], atol=1e-3)
```

Sizning vazifangiz: boshqa kirish uchun h_1

Vazifa

Xuddi shu RNN:

$$W_h = W_x = I_2, \quad b = \mathbf{0}, \quad h_0 = [0, 0].$$

Lekin bu safar kirish:

$$x_1 = [0, 1].$$

Hisoblang:

$$h_1 = ?$$

Eslatma: $h_1 = \tanh(W_h h_0 + W_x x_1 + b)$ formulasidan foydalaning.

Sizning vazifangiz: boshqa kirish uchun h_1

Vazifa

$$W_h = W_x = I_2, \quad b = \mathbf{0}, \quad h_0 = [0, 0], \quad x_1 = [0, 1].$$
$$h_1 = ?$$

Javob

$$\begin{aligned} h_1 &= \tanh(W_h h_0 + W_x x_1 + b) \\ &= \tanh([0, 0] + [0, 1] + [0, 0]) \\ &= [\tanh(0), \tanh(1)] \\ &= [0.000, 0.762]. \end{aligned}$$

Kirish ikkinchi koordinatada edi, shuning uchun vektorli iz ham ikkinchi koordinatada paydo bo'ldi. Birlik matritsada har kirish o'z koordinatasida saqlanadi.

Kod va formula: RNN bir qadami

Python kodi

```
import numpy as np

Wh = np.eye(2)          # W_h = I_2
Wx = np.eye(2)          # W_x = I_2
b  = np.zeros(2)        # b = 0 vektor

def rnn_step(h_prev, x):
    z = Wh @ h_prev + Wx @ x + b
    return np.tanh(z)

h0 = np.zeros(2)
x1 = np.array([1.0, 0.0])

h1 = rnn_step(h0, x1)
print(np.round(h1, 3))  # [0.762 0.   ]
```

Kod va formula mosligi

Kod	Formula
h_prev	h_{t-1}
x	x_t
Wh @ h_prev	$W_h h_{t-1}$
Wx @ x	$W_x x_t$
+ b	$+ b$
np.tanh(z)	$\tanh(\cdot)$

Asosiy formula:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b).$$

PyTorch `nn.RNN` ham shu rekurent yangilash g'oyasiga asoslanadi. Amaliyotda og'irliklar o'qitish orqali o'rganiladi.

RNN tuzog'i: holatni boshlash va tartib muhim

Keng tarqalgan xatolar

- h_0 ni unutib, oldingi gap holatini yangi gapga olib o'tish.
- So'z tartibini buzish: RNN tartibga **sezgir**.
“faqat Ali bugun keldi” $\neq$ “Ali faqat bugun keldi”
- Formula va koddagi aktivatsiyani aralashtirish:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b).$$

To'g'ri amaliy qoida:

- har bir yangi ketma-ketlik uchun $h_0 = \mathbf{0}$ dan boshlang;
- tokenlar tartibini o'zgartirmang;
- koddagi aktivatsiya formulaga mos bo'lsin.

Tartibga sezgirlik — RNNning kuchli tomoni.

Keyingi qadam: bitta RNN qadamini emas, katakni butun ketma-ketlik bo'ylab **yoyib** ko'ramiz.

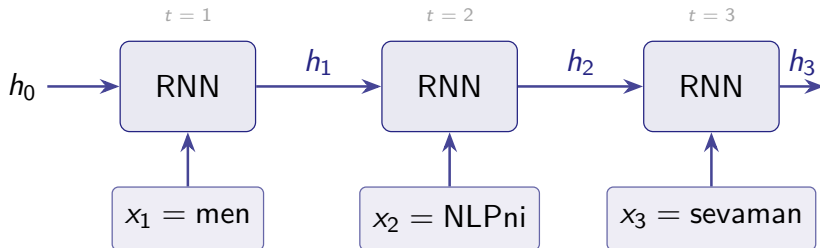
Prezentatsiya rejasi

- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat
- 4 Vaqt bo'ylab yoyilish va BPTT**
- 5 Til modellashtirish va matn tasniflash
- 6 Xulosa va 7-amaliyotga ko'prik

Intuitsiya: bir katakni vaqt bo'ylab yoyamiz

Yoyish (unrolling): bir xil RNN katagi har qadamda **bir xil parametrlar** $\theta = \{W_h, W_x, b\}$ bilan ishlatiladi:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$



Bir xil θ har qadamda ulashiladi — shuning uchun RNN **ixtiyoriy uzunlikdagi** ketma-ketlikni bir xil qoida bilan qayta ishlaydi.

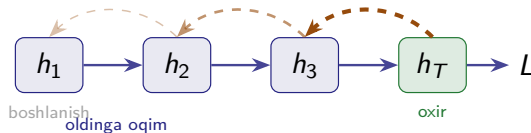
Yoyilish va BPTT: rasmiy ta'rif

$$L = \sum_{t=1}^T L_t$$
$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^T \frac{\partial L_t}{\partial W_h}$$
$$J_t = \frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(1 - h_t^2) W_h$$

BPTT (*Backpropagation Through Time*) — yoyilgan RNN bo'ylab gradientni orqaga tarqatish.

W_h barcha qadamlarda ulashilgani uchun gradient hissalarini vaqt bo'ylab **yig'iladi**.

orqaga gradient J_t^T orqali o'tadi
ko'p qadamda norma $< 1 \Rightarrow$ so'nish



Ko'k — oldinga hisoblash; jigarrang — orqaga gradient.

Uzoq ketma-ketlikda Jacobianlar ko'paytmasi kichrayib, **gradient so'nishi** muammosini keltirishi mumkin.

Gradient nega so'nadi? Asosiy intuitsiya

RNN vaqt bo'ylab yoyilganda, gradient oxirgi qadamdan oldingi qadamlarga qaytadi.

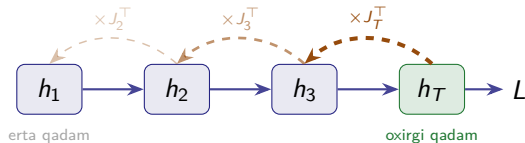
Har orqaga qadamda gradient lokal o'tish matritsasi bilan ko'payadi:

$$\delta_t = J_{t+1}^\top \delta_{t+1}, \quad \delta_t = \frac{\partial L}{\partial h_t}.$$

Muammo: agar har qadamdagi ko'paytiruvchi kichik bo'lsa, gradient tez kichrayadi:

$$0.8^{10} \approx 0.107, \quad 0.8^{30} \approx 0.001.$$

Demak, uzoq boshidagi so'zlarga boradigan gradient juda kichik bo'lib qoladi.



Xulosa: oddiy RNN uzoq bog'liqliklarni o'rganishda qiyinaladi, chunki gradient ko'p qadam davomida qayta-qayta ko'paytiriladi.

Bu keyingi mavzu — LSTM/GRU g'oyasiga olib boradi.

Qo'lda hisob: ikkinchi qadam — $h_2 = [0.642, 0.762]$

Berilganlar:

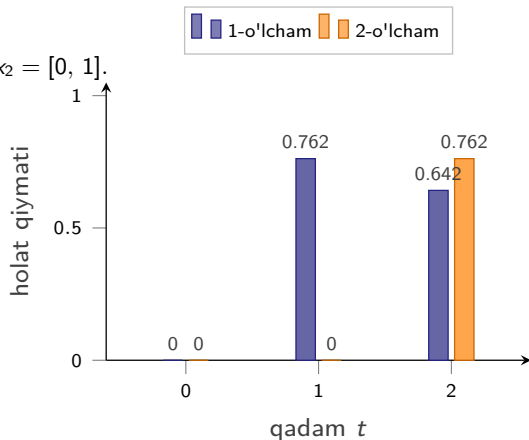
$$W_h = W_x = I_2, \quad b = \mathbf{0}, \quad h_1 = [0.762, 0.000], \quad x_2 = [0, 1].$$

Hisoblaymiz:

$$\begin{aligned} h_2 &= \tanh(W_h h_1 + W_x x_2 + b) \\ &= \tanh([0.762, 0] + [0, 1] + [0, 0]) \\ &= [\tanh(0.762), \tanh(1)] \\ &= [0.642, 0.762]. \end{aligned}$$

$\tanh(0.762) \approx 0.642$, $\tanh(1) \approx 0.762$. Bu soddalashtirilgan misolda h_2 ikkala kirishdan ham **vektorli** iz olib qoladi.

Intuitsiya: h_1 birinchi kirishning izini olib keladi; x_2 ikkinchi kirishni qo'shadi. Shu sababli h_2 oldingi va joriy qadamdan ta'sir oladi.



h_0 bo'sh holat $\rightarrow h_1$ birinchi iz $\rightarrow h_2$ ikki qadamdan ta'sirlangan holat.

Qo'lda hisob: W_h holatni aralashtiradi

Avval $W_h = I_2$ edi. Endi birlik bo'lmagan W_h olamiz:

$$W_h = \begin{pmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{pmatrix}, \quad W_x = I_2, \quad b = \mathbf{0}.$$

Oldingi qadamdan:

$$h_1 = [0.762, 0.000], \quad x_2 = [0, 1].$$

Aralashtirish:

$$W_h h_1 = \begin{pmatrix} 0.5 & -0.5 \\ 0.5 & 0.5 \end{pmatrix} \begin{pmatrix} 0.762 \\ 0 \end{pmatrix} = \begin{pmatrix} 0.381 \\ 0.381 \end{pmatrix}.$$

Yangi holat:

$$\begin{aligned} h_2 &= \tanh(W_h h_1 + W_x x_2 + b) \\ &= \tanh([0.381, 0.381] + [0, 1]) \\ &= \tanh([0.381, 1.381]) \\ &= [\mathbf{0.364}, \mathbf{0.881}]. \end{aligned}$$

Holat	Natija
$W_h = I_2$	$h_2 = [0.642, 0.762]$
W_h aralash	$h_2 = [0.364, 0.881]$

Asosiy fikr:

$W_h = I_2$ bo'lsa, eski iz koordinatalarda alohida qoladi.

Aralashtiruvchi W_h esa h_1 dagi eski izni ikkala koordinataga tarqatadi.

Real RNNda W_h qiymatlari ma'lumotdan o'rganiladi.

Xulosa: W_h eski holatni ko'chiribgina qolmaydi; uni **aralashtirib**, joriy kirish bilan birlashtiradi.

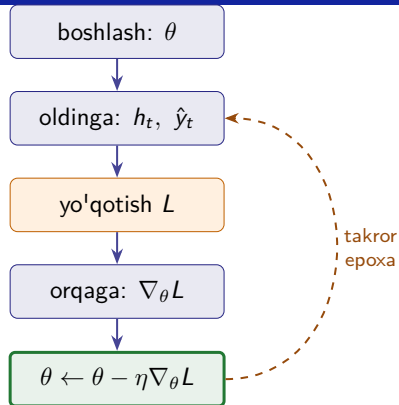
W_h, W_x qayerdan keladi? — o'qitish sikli

Oldingi slaydlarda W_h, W_x ni qo'lda berdik. **Amalda** ular tasodifiy boshlanadi va ma'lumotdan o'rganiladi.

O'qitish sikli

- Boshlash** $\theta = \{W_h, W_x, \dots\}$ kichik tasodifiy qiymatlardan boshlanadi.
- Oldinga** gap bo'ylab h_t va $\hat{y}_t$ hisoblanadi.
- Yo'qotish** bashorat va haqiqiy javob solishtiriladi: $L = \text{cross-entropy}$.
- BPTT** vaqt bo'ylab gradient hisoblanadi: $\nabla_{\theta} L$.
- Yangilash** parametrlar yangilanadi: $\theta \leftarrow \theta - \eta \nabla_{\theta} L$.

Asosiy fikr: model qaysi o'tmishni saqlash va qanday aralashtirishni shu sikl orqali o'zi o'rganadi.



Eslatma: biz W_h, W_x qiymatlarini qo'lda tanlamaymiz; ular gradient orqali bosqichma-bosqich tuzatiladi.

Sizning vazifangiz: tanh hosilasini baholang

Vazifa

BPTT bo'g'inida tanh hosilasi:

$$\tanh'(z) = 1 - \tanh^2(z).$$

Agar holat komponenti $h = 0.762$ bo'lsa:

$$1 - h^2 = ? \quad (\text{bu } < 1 \text{ mi?})$$

O'ylab ko'ring:

- 1 Avval 0.762^2 ni hisoblang.
- 2 Keyin $1 - 0.762^2$ ni toping.
- 3 Natija 1 dan kichikmi?

Agar natija < 1 bo'lsa, ko'p qadamli ko'paytmada gradient kichrayadi.

Sizning vazifangiz: tanh hosilasini baholang

Vazifa

$$h = 0.762 : \quad 1 - h^2 = ?$$

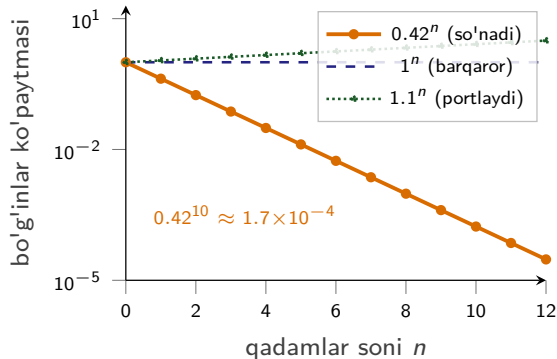
Javob

$$1 - 0.762^2 = 1 - 0.581 = \mathbf{0.419} \approx 0.42.$$

Agar har bo'g'in ≈ 0.42 bo'lsa:

$$0.42^{10} \approx 1.7 \times 10^{-4}.$$

Gradient tez so'nadi.



Intuitsiya: bo'g'in qiymati < 1 bo'lsa, ko'paytma kichrayadi; > 1 bo'lsa, portlash xavfi paydo bo'ladi.

Kod va formula: ketma-ketlik bo'ylab aylanish

Python kodi

```
import numpy as np

Wh, Wx = np.eye(2), np.eye(2)
b = np.zeros(2)

def rnn_forward(xs, h):
    for x in xs:          # t = 1, ..., T
        h = np.tanh(Wh @ h + Wx @ x + b)
    return h

xs = [np.array([1., 0.]),
      np.array([0., 1.])]

hT = rnn_forward(xs, np.zeros(2))
print(np.round(hT, 3))    # [0.642 0.762]
```

Kod va formula mosligi

Kod	Formula
h	h_{t-1}
x	x_t
for x in xs	$t = 1, \dots, T$
Wh @ h	$W_h h_{t-1}$
Wx @ x	$W_x x_t$
+ b	$+ b$
np.tanh(...)	$\tanh(\cdot)$
return h	h_T

Asosiy fikr: for sikli RNNni vaqt bo'ylab yoyadi. Har qadamda bir xil W_h , W_x , b ishlatiladi.

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

Bu misolda $T = 2$, demak $h_T = h_2$.

Yoyilish tuzog'i: so'nuvchi va portlovchi gradient

Muammo: uzoq bog'liqlik so'nadi

RNN vaqt bo'ylab yoyilganda gradient ko'p bo'g'inlar ko'paytmasidan o'tadi:

$$\delta_t = J_{t+1}^\top \delta_{t+1}.$$

Agar ko'p qadam davomida bo'g'inlar normasi < 1 bo'lsa, gradient **so'nadi** (*vanishing gradient*).

Agar normasi > 1 bo'lsa, gradient **portlashi** mumkin (*exploding gradient*).

Natija: oddiy tanh RNN uzoq masofadagi bog'liqliklarni o'rganishda qiynalishi mumkin.

Amaliy yechimlar

- **Gradient clipping:** portlovchi gradientni chegaralaydi.
- **LSTM** (*Long Short-Term Memory*) va **GRU** (*Gated Recurrent Unit*): darvozalar orqali uzoq xotira muammosini yumshatadi.

O'zbek tili bilan bog'lanish:

O'zbek tilida kesim ko'pincha gap oxirida keladi (SOV: ega-to'ldiruvchi-kesim). Shuning uchun uzoqdagi so'zlar orasidagi bog'liqlikni ushlash muhim.

Keyingi g'oya: LSTM/GRU oddiy RNNdagi gradient so'nishini kamaytirishga yordam beradi.

Prezentatsiya rejasi

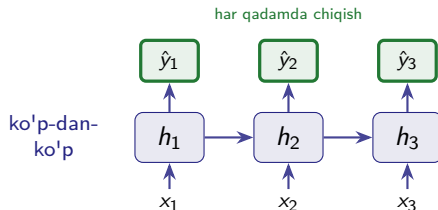
- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat
- 4 Vaqt bo'ylab yoyilish va BPTT
- 5 Til modellashtirish va matn tasniflash**
- 6 Xulosa va 7-amaliyotga ko'prik

Intuitsiya: bir xil RNN yadrosi, ikki vazifa

Ko'p-dan-ko'p (*many-to-many*):

har qadamda keyingi so'z bashorat qilinadi. Har bir h_t ustidan chiqish olinadi:

$$\hat{y}_t = \text{softmax}(W_y h_t + b_y).$$

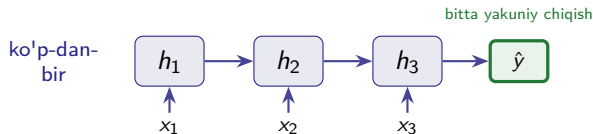


Ko'p-dan-bir (*many-to-one*):

butun gap o'qiladi, keyin oxirgi holat h_T dan bitta sinf bashorat qilinadi:

$$\hat{y} = \text{softmax}(W_c h_T + b_c).$$

Masalan: ijobiy/salbiy.



Yuqori: har bir h_t dan chiqish. Past: faqat oxirgi holat h_T dan chiqish.

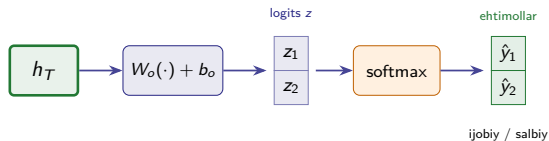
RNN chiqish qatlami: rasmiy ta'rif

$$z = W_o h_T + b_o, \quad \hat{y} = \text{softmax}(z)$$

Belgilar:

- h_T — oxirgi yashirin holat, ya'ni gapning vektorli xulosasi;
- W_o, b_o — chiqish qatlamining o'rganiladigan parametrlari;
- z — sinf ballari (*logits*);
- $\hat{y}$ — sinflar ehtimoli, masalan ijobiy/salbiy.

Eslatma: matn tasniflashda odatda oxirgi holat h_T dan bitta chiqish olinadi. Til modelida esa har bir h_t ustidan keyingi so'z taqsimoti hisoblanadi.



Oxirgi holat $\rightarrow$ logits $\rightarrow$ ehtimollar.

Keltirib chiqarish: nega oxirgi holat h_T ?

1-qadam. h_T rekursiya orqali $x_1, \dots, x_T$ dan ta'sir olgan **yakuniy yashirin holat**dir.

2-qadam. Tasniflash uchun **bitta** vektor kerak; h_T o'zgaruvchan uzunlikdagi gapni o'zgarmas o'lchamli vektorga jamlaydi.

3-qadam. Shu vektorni chiqish qatlami va softmax orqali sinf ehtimollariga aylantiramiz:

$$\hat{y} = \text{softmax}(W_o h_T + b_o).$$

Xulosa: RNN matnni yakuniy holat h_T orqali vektorli xulosa sifatida ifodalaydi; keyin shu vektor chiqish qatlamiga beriladi.

Qo'lda hisob: oxirgi holatdan sentiment sinfiga

Berilganlar:

$$w_{ijobiy} = [1, 0], \quad w_{salbiy} = [0, 1], \quad b_o = 0.$$

$$h_T = [1, 0] \quad (\text{soddalashtirilgan o'quv misoli}).$$

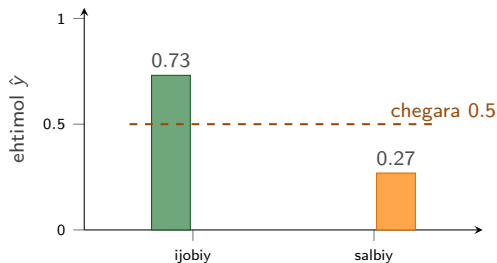
Ballar:

$$z_{ijobiy} = [1, 0] \cdot [1, 0] = 1, \quad z_{salbiy} = [0, 1] \cdot [1, 0] = 0.$$

Softmax:

$$\hat{y}_{ijobiy} = \frac{e^1}{e^1 + e^0} \approx \mathbf{0.731}, \quad \hat{y}_{salbiy} \approx 0.269.$$

Xulosa: $0.731 > 0.5$, demak model bashorati **ijobiy**.
RNN oxirgi holat h_T orqali sinfni aniqlaydi.



$$z = [1, 0] \xrightarrow{\text{softmax}} \hat{y} = [0.731, 0.269] \text{ — } ijobiy \text{ ehtimoli katta.}$$

Sizning vazifangiz: boshqa holat uchun bashorat

Vazifa

Agar $h_T = [0, 1]$ bo'lsa va chiqish og'irliklari oldingidek bo'lsa:

$\hat{y}_{ijobiy} = ?$ bashorat qaysi sinf?

Sizning vazifangiz: boshqa holat uchun bashorat

Vazifa

$$h_T = [0, 1] : \quad \hat{y}_{ijobiy} = ?$$

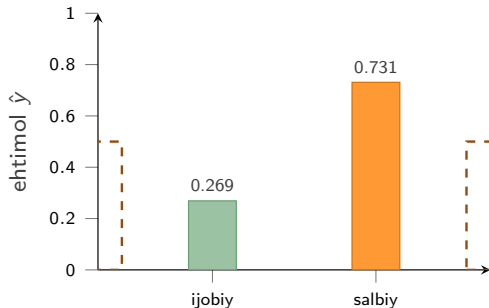
Javob

$$z_{ijobiy} = [1, 0] \cdot [0, 1] = 0, \quad z_{salbiy} = [0, 1] \cdot [0, 1] = 1.$$

$$\hat{y}_{ijobiy} = \frac{e^0}{e^0 + e^1} \approx \mathbf{0.269}.$$

$$\hat{y}_{salbiy} \approx 0.731 \Rightarrow \mathbf{salbiy}.$$

Oxirgi holatda ikkinchi koordinata ustun bo'lgani uchun salbiy sinf ehtimoli katta.



Endi salbiy sinf ustun: $0.731 > 0.5$.

Kod va formula: PyTorch nn.RNN

Python kodi

```
import torch.nn as nn

class RNNClassifier(nn.Module):
    def __init__(self, vocab, dim, hid, n_cls):
        super().__init__()
        self.emb = nn.Embedding(vocab, dim)
        self.rnn = nn.RNN(dim, hid, batch_first=True)
        self.out = nn.Linear(hid, n_cls)

    def forward(self, x):
        e = self.emb(x)          # token -> embedding
        _, h = self.rnn(e)       # h[-1]: oxirgi holat  $h_T$ 
        z = self.out(h[-1])      # logits:  $W_o h_T + b_o$ 
        return z
```

Kod va formula mosligi

Kod	Formula
nn.Embedding	x_t yoki e_t
nn.RNN	$h_t = \tanh(\cdot)$
h[-1]	h_T
nn.Linear	$z = W_o h_T + b_o$
return z	logits z

Eslatma: bu model oxirgi holat h_T dan **logits** chiqaradi. Ehtimollar softmax orqali olinadi:

$$\hat{y} = \text{softmax}(z).$$

7-amaliyotdagi RNNClassifier shu asosiy arxitekturaga tayanadi.

Qo'llash tuzog'i: padding va o'zgaruvchan uzunlik

Batch ichida uzunlik har xil

Gaplar har xil uzunlikda bo'ladi; batch tuzish uchun **padding** (to'ldirish tokeni) qo'shiladi.

Lekin padding qadamlarini haqiqiy so'z deb qayta ishlasak, oxirgi holat h_T buzilishi mumkin.

Yechim:

`pack_padded_sequence` (PyTorch)
yoki **masking** (niqoblash) ishlatiladi.

Maqsad: paddingdan keyingi h_T emas,
balki haqiqiy oxirgi holatni olish:

$$h_{\ell_i},$$

bu yerda ℓ_i — i -gapning haqiqiy uzunligi.

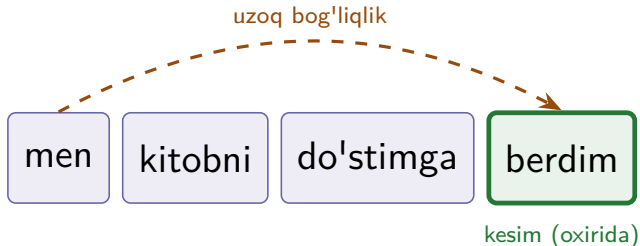
Bu 7-amaliyotda qo'shimcha sifatida beriladi.

Prezentatsiya rejasi

- 1 Kirish: ketma-ketlikni xotira bilan modellaymiz
- 2 O'zgarmas kontekst oynasidan ketma-ket xotiraga
- 3 RNN konsepsiyasi: yashirin holat
- 4 Vaqt bo'ylab yoyilish va BPTT
- 5 Til modellashtirish va matn tasniflash
- 6 Xulosa va 7-amaliyotga ko'prik

O'zbek tilida: SOV tartibi va uzun qo'shimchalar RNN ni sinaydi

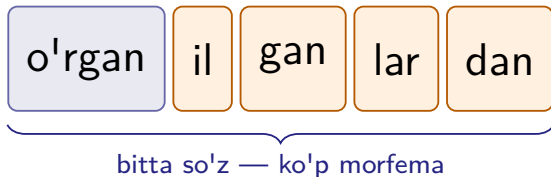
1. SOV — kesim oxirida:



Oqibat

So'z-darajali RNN uzoq zanjir va SOV kesimini ushlashi kerak — ammo so'nuvchi gradient buni qiyinlashtiradi.

2. Uzun qo'shimcha zanjiri:



Yechim: LSTM/GRU (8-ma'ruza), subword/belgi-darajali yondashuvlar yoki morfologik normalizatsiya.

Sintez: o'zgarmas oynadan rekurrent xotiraga

Model	Kontekst	Parametr	Cheklov
N-gram (5-m.)	o'zgarmas $n-1$	lug'at n (siyrak)	uzoq bog'liqlik yo'q
Oldinga uzatuvchi LM	o'zgarmas k	$k \cdot d$ ga bog'liq	chegaralangan oyna
RNN (bugun)	ixtiyoriy uzunlik	ulashilgan W_h, W_x	so'nuvchi gradient

Asosiy g'oya

RNN o'tmishni o'zgarmas o'lchamli holatga jamlaydi va har qadamda bir xil og'irliklar bilan yangilaydi $\Rightarrow$ ixtiyoriy uzunlik. Narxi: uzoq bog'liqlikda gradient so'nadi (8-ma'ruza buni hal qiladi).

Seminal manba: Rumelhart, Hinton, Williams (1986)

Rumelhart, D.E., Hinton, G.E., & Williams, R.J. (1986). **Learning Representations by Back-Propagating Errors.** *Nature*, 323, 533–536.

Ahamiyati

- Orqaga tarqatish (*backpropagation*) ni mashhur qildi — chuqur va rekurrent tarmoqlar asosi.
- BPTT (*Backpropagation Through Time*) — shu g'oyaning vaqt bo'ylab yoyilgan ko'rinishi.
- Zamonaviy NLP modellari (RNN → LSTM/GRU → Transformer) o'qitish ildizi.

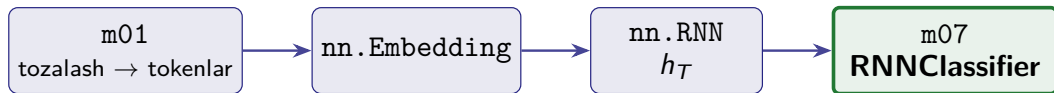
Muhokama savoli

Agar gradient uzoq qadamlarda so'lsa, oddiy RNN “men ...**sevaman**” dagi uzoq bog'liqlikni qanday o'rganadi? Bu cheklovni qanday yengish mumkin?)

Bugungi maqsadlar bajarildi

- 1 ✓ **Tushuntirdik:** nega o'zgarmas oyna (N-gram / oldinga uzatuvchi) uzoq bog'liqlikni ushlay olmaydi.
- 2 ✓ **Hisobladik:** RNN bir qadami — $h_1 = \tanh([1, 0]) = [0.762, 0.000]$.
- 3 ✓ **Tushuntirdik:** yoyilish, og'irliklar ulashilishi va BPTT ($h_2 = [0.642, 0.762]$; so'nuvchi gradient).
- 4 ✓ **Qo'lladik:** oxirgi holatdan tasniflash — softmax → ijobiy (0.731).

7-amaliyot: RNN bilan matn tasniflagich quriladi



- RNNClassifier: Embedding + nn.RNN + Linear;
- matnni ijobiy/salbiy tasniflash;
- natijani F1-baho bilan tekshirish.

Birinchi tekshiruv

Ma'ruzaqadagi qo'lda hisoblangan RNN bir qadami:

$$h_1 \approx [0.762, 0.0].$$

```
assert np.allclose(  
    h1, [0.762, 0.0],  
    atol=1e-3)
```

8-ma'ruzaga ko'prik: holatni qayta yozish vs darvozalar

Oddiy RNN har qadamda yashirin holatni **to'liq qayta yozadi**:

$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b).$$

Shuning uchun uzoq bog'liqlikda gradient **so'nadi** (*vanishing*) — model gap boshidagi muhim so'zni barqaror eslab qola olmaydi.

Darvozalar (*gates*) **nimani saqlash** va **nimani unutishni** o'rganadi — muhim ma'lumot uzoq saqlanadi, ortiqchasi o'chiriladi.

Ya'ni: “hammasini qayta yozish”dan → “tanlab saqlash/unutish”ga.

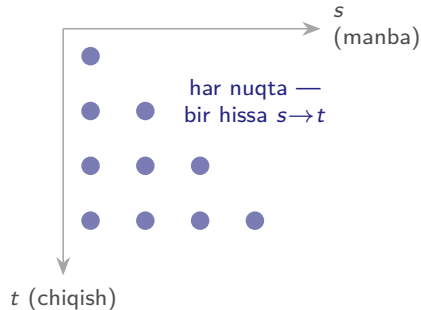
- ❶ Rumelhart, D.E., Hinton, G.E., & Williams, R.J. (1986). *Learning Representations by Back-Propagating Errors*. Nature, 323, 533–536.
- ❷ Elman, J.L. (1990). *Finding Structure in Time*. Cognitive Science, 14(2). (Oddiy rekurrent tarmoq.)
- ❸ Werbos, P.J. (1990). *Backpropagation Through Time*. Proc. IEEE, 78(10).
- ❹ Jurafsky, D., & Martin, J. H. (2023). *Speech and Language Processing* (3-nashr). 9-bob: RNN va LSTM.
- ❺ `torch.nn.RNN` hujjatlari — PyTorch rekurrent qatlamlar.

Ilova S1: BPTT — gradientlarning yig'ilishi

W_h barcha qadamlarda ulashilgani uchun to'liq gradient har qadam hissasining yig'indisi:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^T \sum_{s=1}^t \frac{\partial L_t}{\partial h_t} \left(\prod_{r=s+1}^t \frac{\partial h_r}{\partial h_{r-1}} \right) \frac{\partial h_s}{\partial W_h}$$

ichki ko'paytma — s dan t gacha bo'lgan zanjir; aynan shu ko'paytma uzoq masofada so'nadi yoki portlaydi.



Ikki yig'indi: chiqish t va manba $s \leq t$ bo'yicha.

Amalda: gradient kesish (*clipping*) portlashni, LSTM/GRU darvozalari so'nishni yumshatadi.

Ilova S2: gradient nega Jacobianlar ko'paytmasi?

Holatlar vaqt bo'ylab bog'langan:

$$h_t \rightarrow h_{t+1} \rightarrow \dots \rightarrow h_T.$$

Shuning uchun:

$$\frac{\partial h_T}{\partial h_t} = J_T J_{T-1} \dots J_{t+1}, \quad J_s = \frac{\partial h_s}{\partial h_{s-1}}.$$

Orqaga gradient:

$$\frac{\partial L}{\partial h_t} = J_{t+1}^\top \dots J_T^\top \frac{\partial L}{\partial h_T}.$$

2-qadam: RNN Jacobiani

RNN qadami:

$$h_s = \tanh(W_h h_{s-1} + W_x x_s + b).$$

Demak:

$$J_s = \frac{\partial h_s}{\partial h_{s-1}} = \text{diag}(1 - h_s^2) W_h.$$

Bu yerda $1 - h_s^2$ — $\tanh$ hosilasi.

3-qadam: norma

$$\|J_T \dots J_{t+1}\| \leq \prod_{s=t+1}^T \|J_s\|.$$

Agar ko'p qadamda $\|J_s\| < 1$ bo'lsa, gradient **so'nadi**;
 $\|J_s\| > 1$ bo'lsa, **portlashi** mumkin.

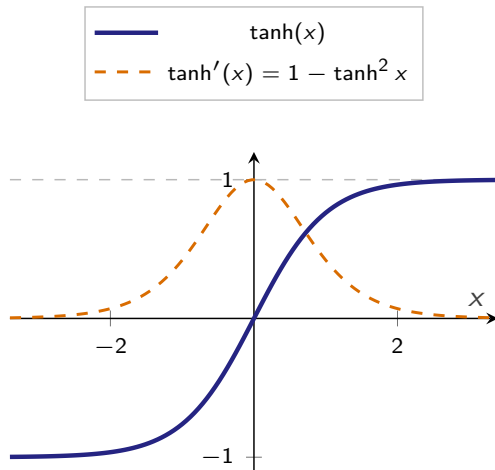
Ilova S3: nega aynan tanh?

tanh chiqishni $[-1, 1]$ ga **siqadi** — holat qiymatlari portlamaydi va markazlashgan (0 atrofida), bu o'qitishni barqarorlashtiradi.

$$\tanh'(x) = 1 - \tanh^2(x) \in (0, 1]$$

hosila ≤ 1 bo'lgani uchun ko'p qadam ko'paytmasi so'nishga moyil — so'nuvchi gradientning matematik ildizi.

Muqobil: ReLU tez, ammo rekurrentda beqaror bo'lishi mumkin; shuning uchun klassik RNN tanh ishlatadi.



Hosila $x=0$ da eng katta ($= 1$), dumlarda $\rightarrow 0$.